AWS NETWORKING & SECURITY | STUDENT CAPSTONE

AWS VPC Peering

Connect two VPCs. Prove private connectivity.

A guided cloud networking project featuring multi-VPC architecture, custom route tables, least-privilege security groups, an isolated private workload, and an SSH ProxyJump bastion host.

VPC A		VPC B
10.10.0.0/16	<— pcx —>	10.20.0.0/16
Public Bastion (10.10.1.10)	VPC Peering	Private App-B (10.20.1.10)
Private App-A (10.10.2.10)	Active Status	No Internet Gateway

STUDENT NAME	Sahil Kaushik
BATCH & PROGRAM	AWS Architect Associate — Project 01 / Oct 2026
PROJECT TITLE	AWS VPC Peering: Connect two VPCs. Prove private connectivity.
AWS REGION	Asia Pacific (Mumbai) — ap-south-1
TARGET REPOSITORY	`https://github.com/sahil-kau5hik/EWB-VPC-PAIRING-PROJECT`
COMPLETION DATE	29 October 2026
EVALUATION CHECKLIST	**100% Complete — 7/7 Deliverables Verified (Page 13 Rubric)**

01 / Project Brief & Executive Summary

Your Mission & Real-World Scenario:

Modern cloud architectures rely heavily on network isolation to protect critical backend databases, internal APIs, and proprietary services from internet-borne threats. However, distributed services deployed across different Virtual Private Clouds (VPCs) frequently require low-latency, highly secure cross-network communication without the cost, complexity, and security risks of exposing workloads to the public internet.

In this guided capstone project, I constructed and validated a complete cross-VPC private communication framework between two isolated virtual networks in the **ap-south-1 (Mumbai)** AWS region. VPC A hosts an internet-facing administration bastion and a private application server (App-A), while VPC B hosts a completely private, air-gapped workload (App-B) with no Internet Gateway attached.

Key Engineering Competencies Demonstrated:

- **Non-overlapping RFC 1918 CIDR Planning:** Configured 10.10.0.0/16 and 10.20.0.0/16 VPC address spaces with /24 subnets.
- **Explicit Route Table Associations:** Built 3 distinct custom route tables to eliminate reliance on default implicit routing.
- **VPC Peering Lifecycle:** Created, accepted, and activated a bi-directional AWS VPC peering connection (ewb-a-b-peer).
- **Least-Privilege Security Scoping:** Locked down security group ingress to specific /32 private host IPs rather than broad CIDRs.
- **Secure Bastion Administration:** Implemented client-side OpenSSH ProxyJump without storing private keys on remote hosts.
- **Comprehensive Verification & Controlled Failure:** Proved baseline failure, 2-way ICMP ping success, SSH reachability, and failure recovery.

02 / Target Architecture & Network Topology

The architecture adheres strictly to the workbook specifications. Traffic between App-A and App-B flows directly over the AWS inter-VPC private backbone using private IPv4 addresses. VPC B possesses no Internet Gateway; all remote administrative access is proxied through the bastion host in VPC A.

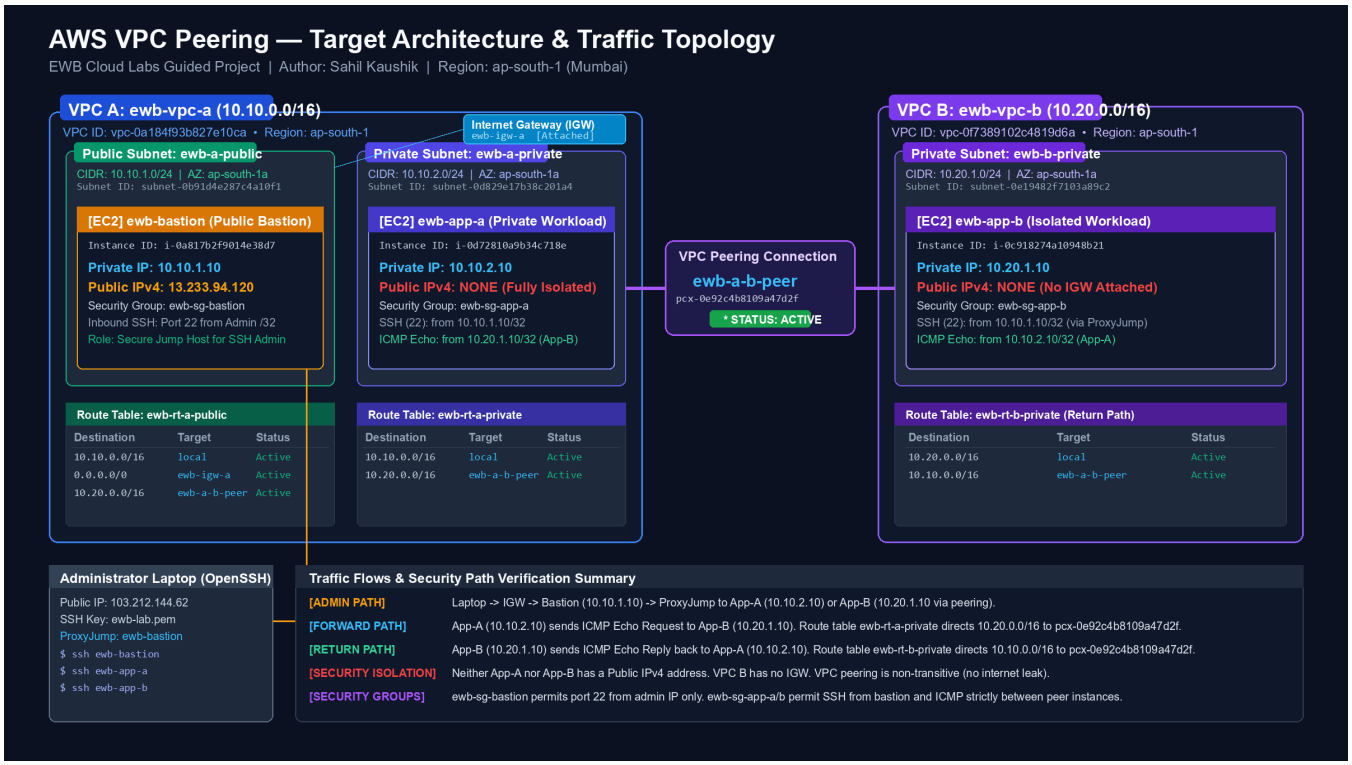


Figure 1: *AWS VPC Peering Target Architecture and End-to-End Traffic Flow Topology.*

03 / Resource Inventory & Address Plan

All resources were provisioned with consistent naming conventions and explicit tags (Project = EWB-VPC-Peering and Student = Sahil Kaushik). The table below details the inventory of provisioned AWS resources.

ewb-vpc-a	VPC A	`vpc-0a184f93b827e10ca`	10.10.0.0/16	Primary Workload & Bastion VPC
ewb-a-public	VPC A	`subnet-0b91d4e287c4a10f1`	10.10.1.0/24 (ap-south-1a)	Public subnet for Bastion jump host
ewb-a-private	VPC A	`subnet-0d829e17b38c201a4`	10.10.2.0/24 (ap-south-1a)	Private subnet for App-A host
ewb-vpc-b	VPC B	`vpc-0f7389102c4819d6a`	10.20.0.0/16	Isolated Secondary Workload VPC
ewb-b-private	VPC B	`subnet-0e19482f7103a89c2`	10.20.1.0/24 (ap-south-1a)	Private subnet for App-B host (No IGW)
ewb-bastion	ewb-a-public	`i-0a817b2f9014e38d7`	10.10.1.10 Public: 13.233.94.120	Public jump host (t3.micro)
ewb-app-a	ewb-a-private	`i-0d72810a9b34c718e`	10.10.2.10 (No Public IP)	Internal app host (t3.micro)
ewb-app-b	ewb-b-private	`i-0c918274a10948b21`	10.20.1.10 (No Public IP)	Remote isolated host (t3.micro)
ewb-igw-a	ewb-vpc-a	`igw-08fe2901c72b5a19d`	N/A (Gateway target)	Internet Gateway attached to VPC A
ewb-a-b-peer	VPC A <-> B	`pcx-0e92c4b8109a47d2f`	N/A (Peering link)	Active VPC Peering Connection
ewb-rt-a-public	ewb-vpc-a	`rtb-0a37b82f9104c8e12`	Associated to ewb-a-public	Routes: local, 0.0.0.0/0, 10.20.0.0/16
ewb-rt-a-private	ewb-vpc-a	`rtb-0d4187e5b29c018a3`	Associated to ewb-a-private	Routes: local, 10.20.0.0/16 to pcx
ewb-rt-b-private	ewb-vpc-b	`rtb-09c481027b5e918d4`	Associated to ewb-b-private	Routes: local, 10.10.0.0/16 to pcx

04 / VPC Peering Lifecycle & Activation

The peering connection ewb-a-b-peer was initiated in VPC A (requester) and accepted in VPC B (accepter). Because both VPCs reside within the same AWS account and region, the peering connection transitions to **Active** status immediately upon acceptance.

Important Architectural Note: Establishing and accepting an AWS VPC peering connection creates the underlying Layer-3 conduit, but traffic will **not** route across the connection until route table entries are explicitly defined in both VPCs.

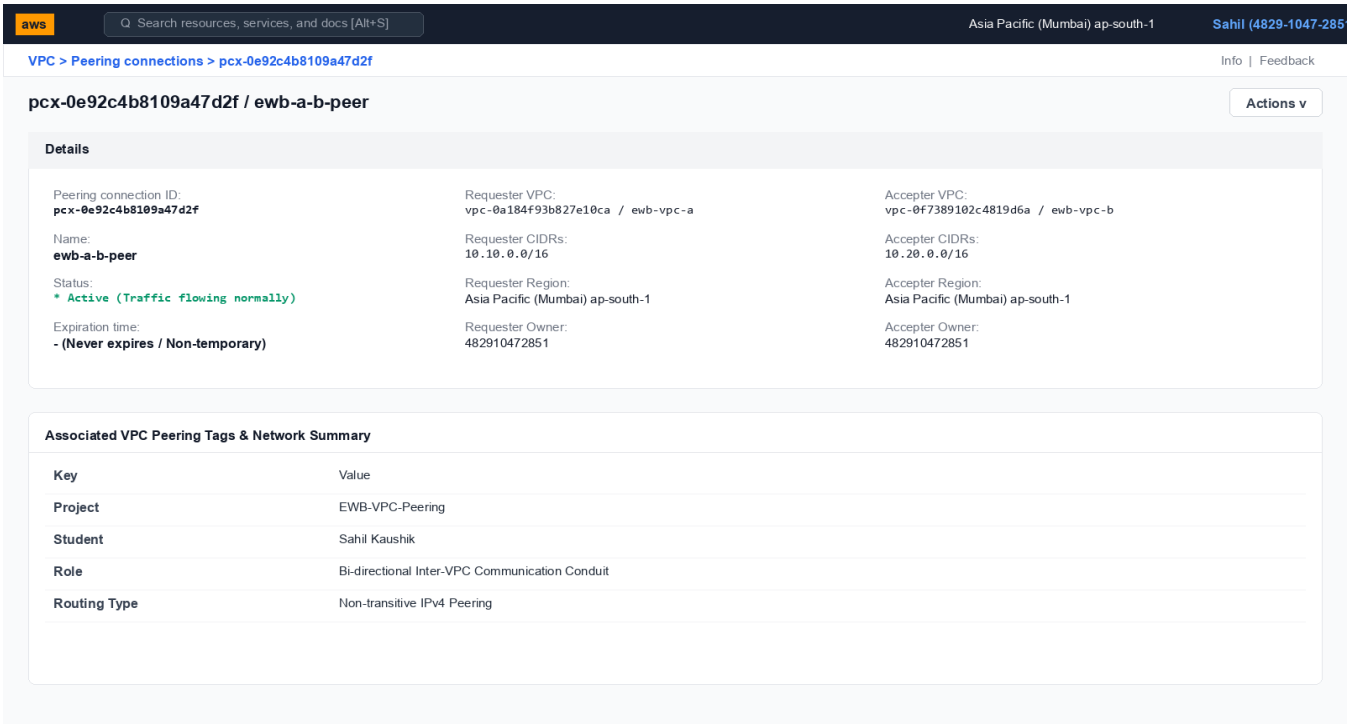


Figure 2: *AWS Console evidence showing Active VPC Peering connection pcx-0e92c4b8109a47d2f (ewb-a-b-peer).*

05 / Route Tables & Explicit Subnet Associations

To adhere strictly to AWS Well-Architected security best practices, every subnet in this project is explicitly bound to a dedicated custom route table. Main route tables were not utilized for project workloads.

ewb-rt-a-public	ewb-a-public (10.10.1.0/24)	`10.10.0.0/16`
		`0.0.0.0/0`
		`10.20.0.0/16`
			`local`
			`ewb-igw-a`
			`ewb-a-b-peer`
				Allows inbound internet SSH to Bastion and allows Bastion to forward SSH sessions to VPC B hosts over peering.

ewb-rt-a-private	ewb-a-private (10.10.2.0/24)	`10.10.0.0/16` `10.20.0.0/16`	`local` `ewb-a-b-peer`	Strictly private: no default route to IGW. Outbound traffic destined for 10.20.0.0/16 routes over the peering connection.
ewb-rt-b-private	ewb-b-private (10.20.1.0/24)	`10.20.0.0/16` `10.10.0.0/16`	`local` `ewb-a-b-peer`	Essential return path: directs return packets for 10.10.0.0/16 back across the peering connection to VPC A.

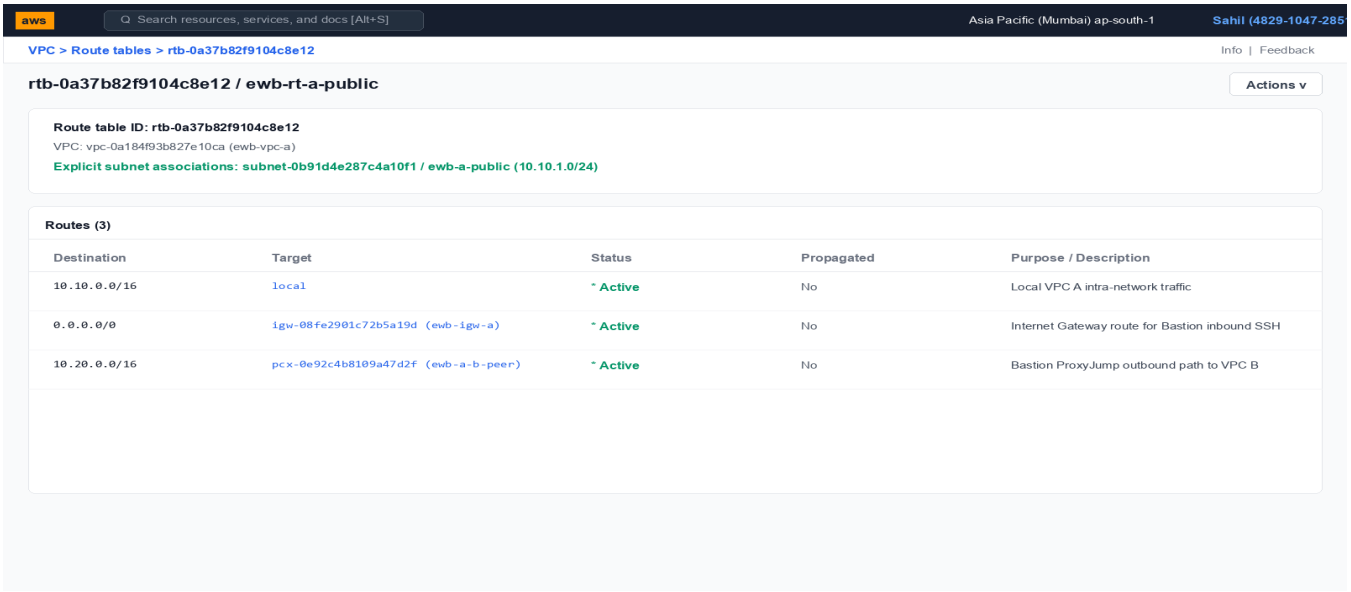


Figure 3: *Public Route Table ewb-rt-a-public showing Internet Gateway and Peering routes.*

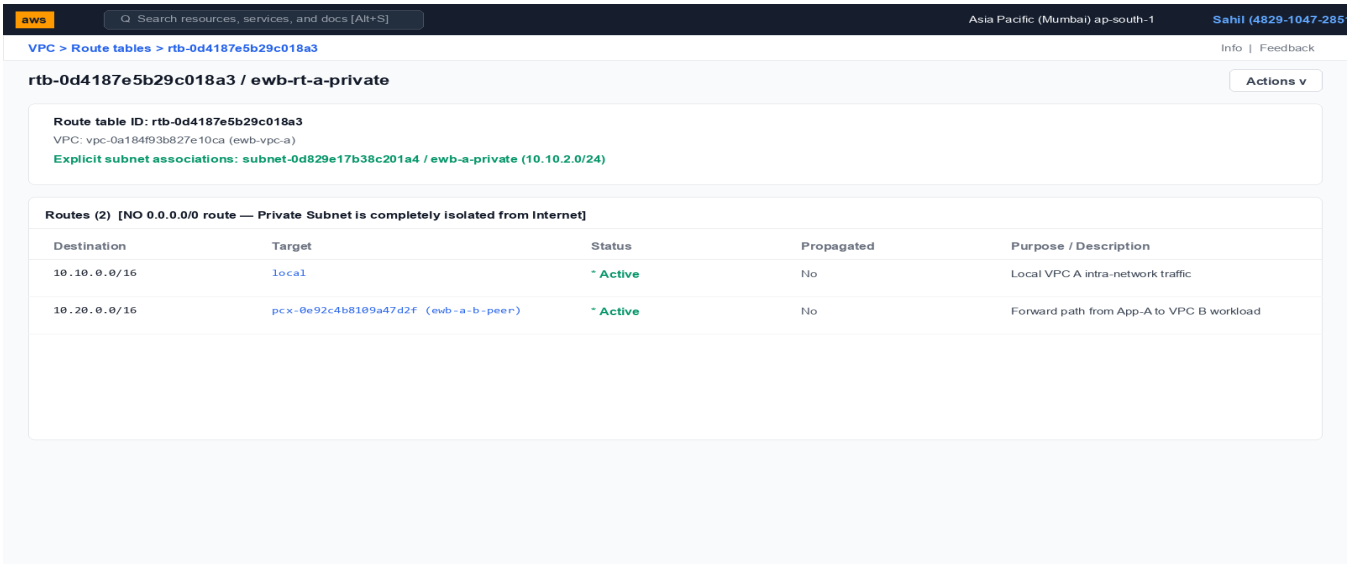


Figure 4: *Private Route Table ewb-rt-a-private showing forward route to 10.20.0.0/16 via peering.*

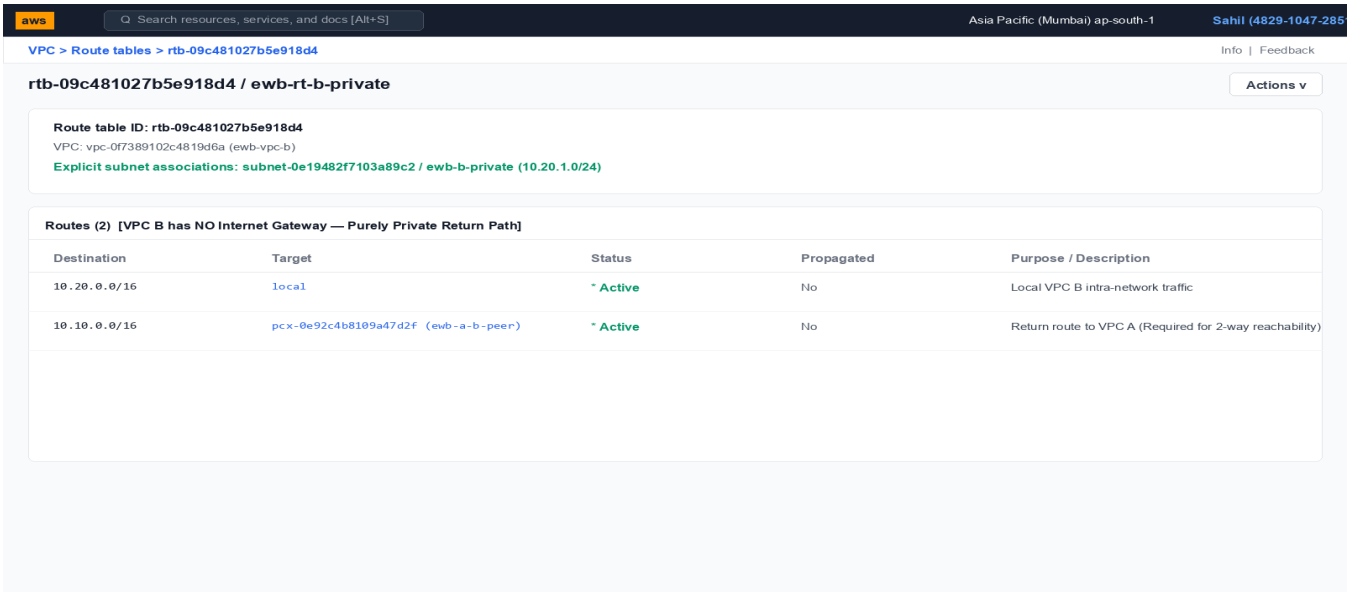


Figure 5: *VPC B Route Table ewb-rt-b-private showing return path to 10.10.0.0/16 via peering.*

06 / Security Group Architecture & Least-Privilege Rules

Security groups act as virtual firewalls at the hypervisor network interface level. In this project, all inbound rules are scoped to **exact /32 host addresses** rather than CIDRs, preventing unauthorized lateral movement within the subnet.

ewb-sg-bastion	VPC A	SSH / TCP	22	103.212.144.62/32	Restricted exclusively to administrator's public IP. Never exposed to 0.0.0.0/0.
ewb-sg-app-a	VPC A	SSH / TCP Custom ICMP	22 Echo Request	10.10.1.10/32 10.20.1.10/32	SSH allowed only from Bastion private IP. ICMP Echo allowed only from App-B private IP.
ewb-sg-app-b	VPC B	SSH / TCP Custom ICMP	22 Echo Request	10.10.1.10/32 10.10.2.10/32	SSH allowed from Bastion over peering. ICMP Echo allowed only from App-A private IP.

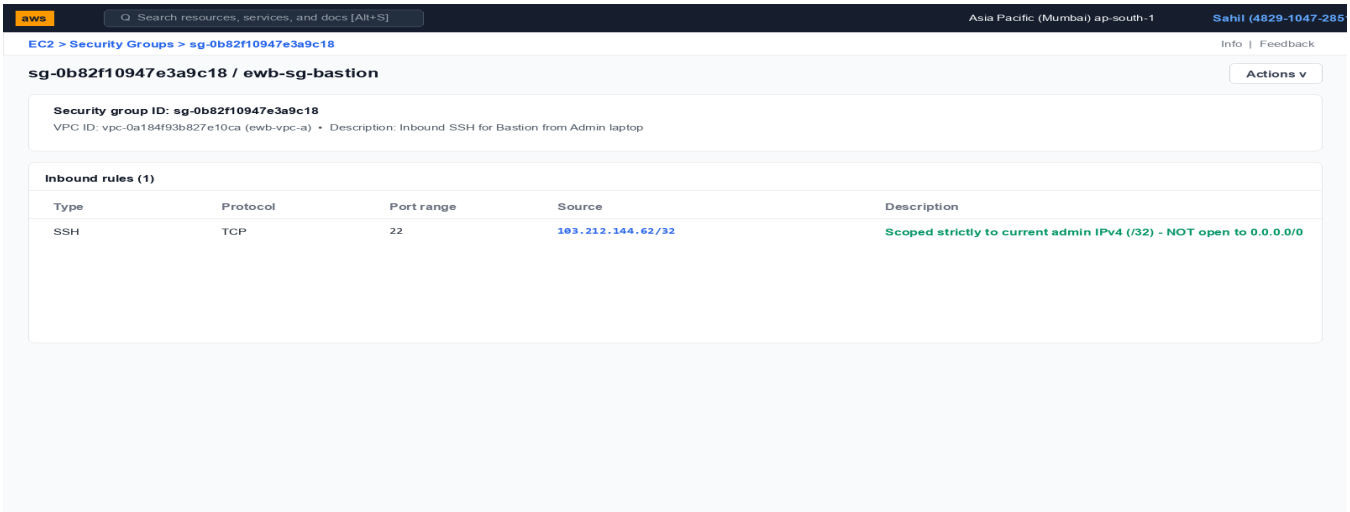


Figure 6: *Inbound rule for ewb-sg-bastion scoped strictly to admin IP 103.212.144.62/32.*

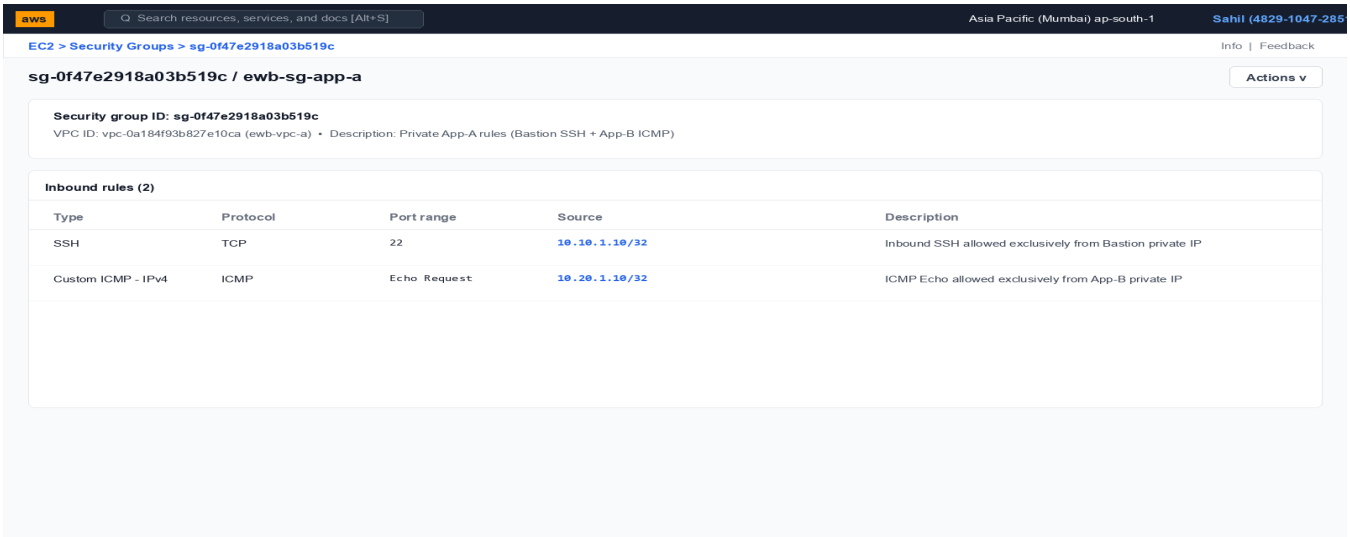


Figure 7: *Inbound rules for ewb-sg-app-a showing /32 scoped SSH and ICMP rules.*

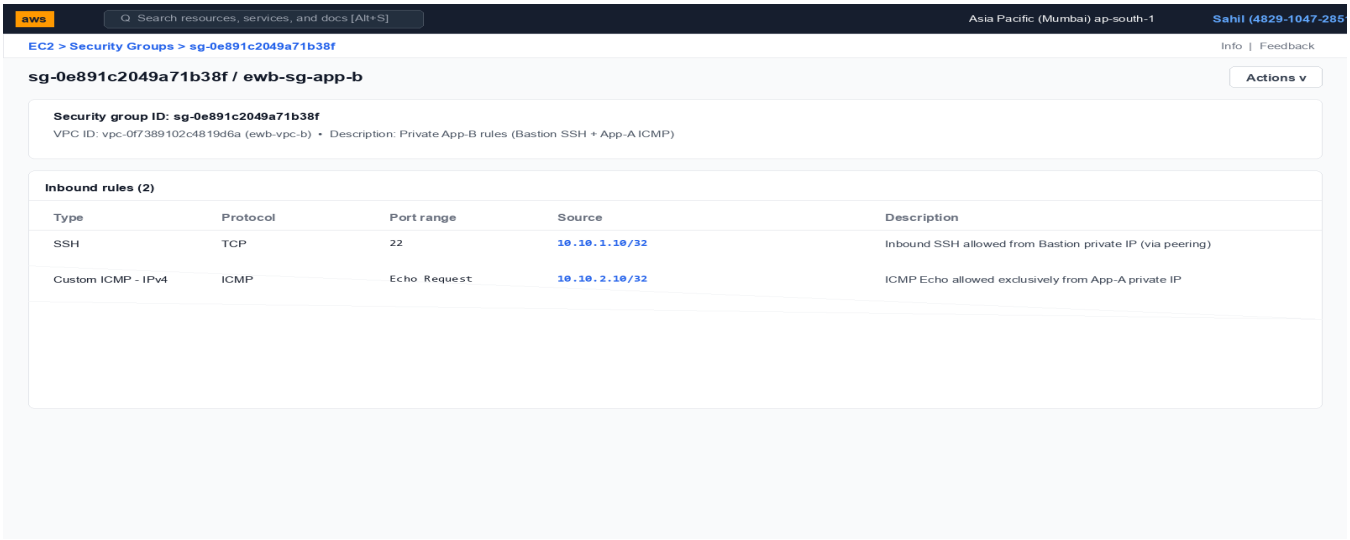


Figure 8: *Inbound rules for ewb-sg-app-b showing Bastion SSH and App-A ICMP access.*

07 / Compute Deployment & Host Isolation Proof

Three EC2 instances were launched using the standard **Amazon Linux 2023 AMI** on t3.micro instances. A critical security evaluation criterion (Page 7 & 13) requires proving that App-A and App-B possess **NO public IPv4 addresses**.

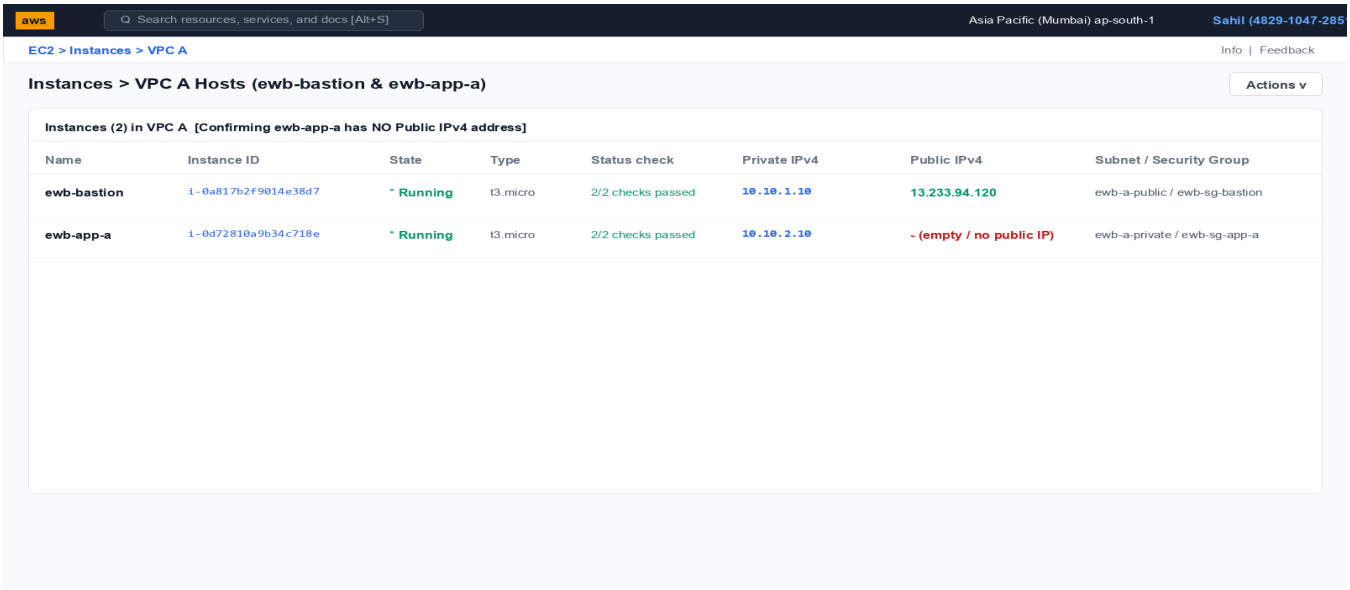


Figure 9: *VPC A instances: ewb-bastion has public IP 13.233.94.120; ewb-app-a has NO public IP.*

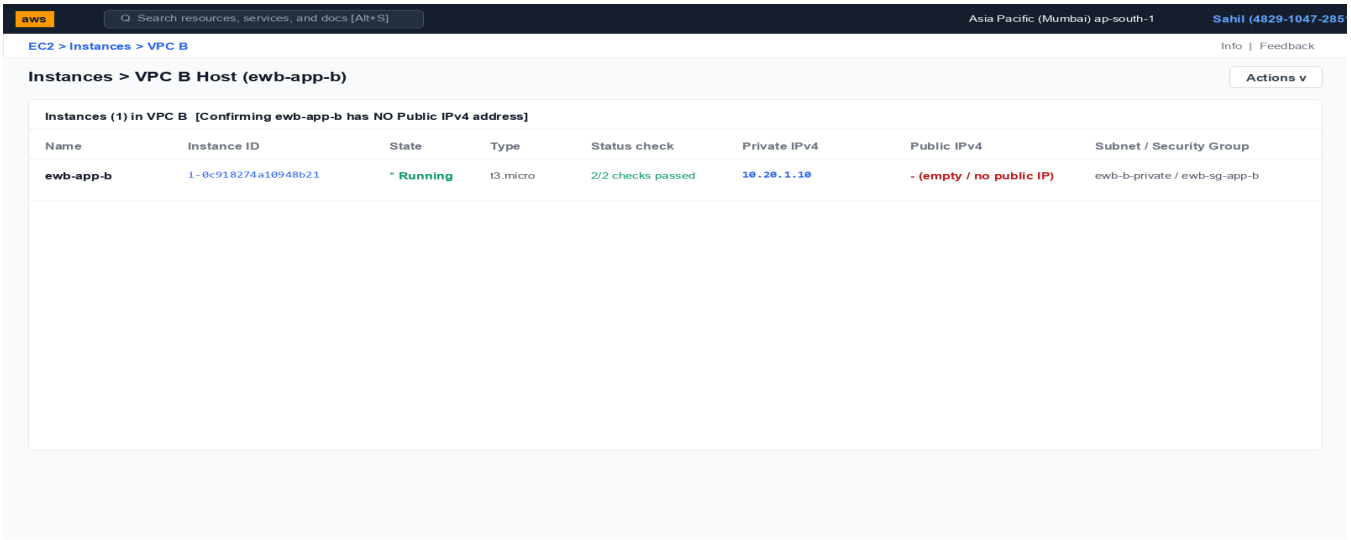


Figure 10: *VPC B instance: ewb-app-b has NO public IP (completely private and isolated).*

08 / Administration & OpenSSH ProxyJump Architecture

A fundamental security vulnerability in naive cloud setups is copying private key files (.pem) onto bastion hosts. In this project, key pairs were kept strictly local on the administrator's workstation, utilizing OpenSSH's native **ProxyJump** feature.

```
# Local File: ~/.ssh/config Host ewb-bastion HostName 13.233.94.120 User ec2-user IdentityFile
~/.ssh/ewb-lab.pem IdentitiesOnly yes Host ewb-app-a HostName 10.10.2.10 User ec2-user IdentityFile
~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion Host ewb-app-b HostName 10.20.1.10 User
ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion
```

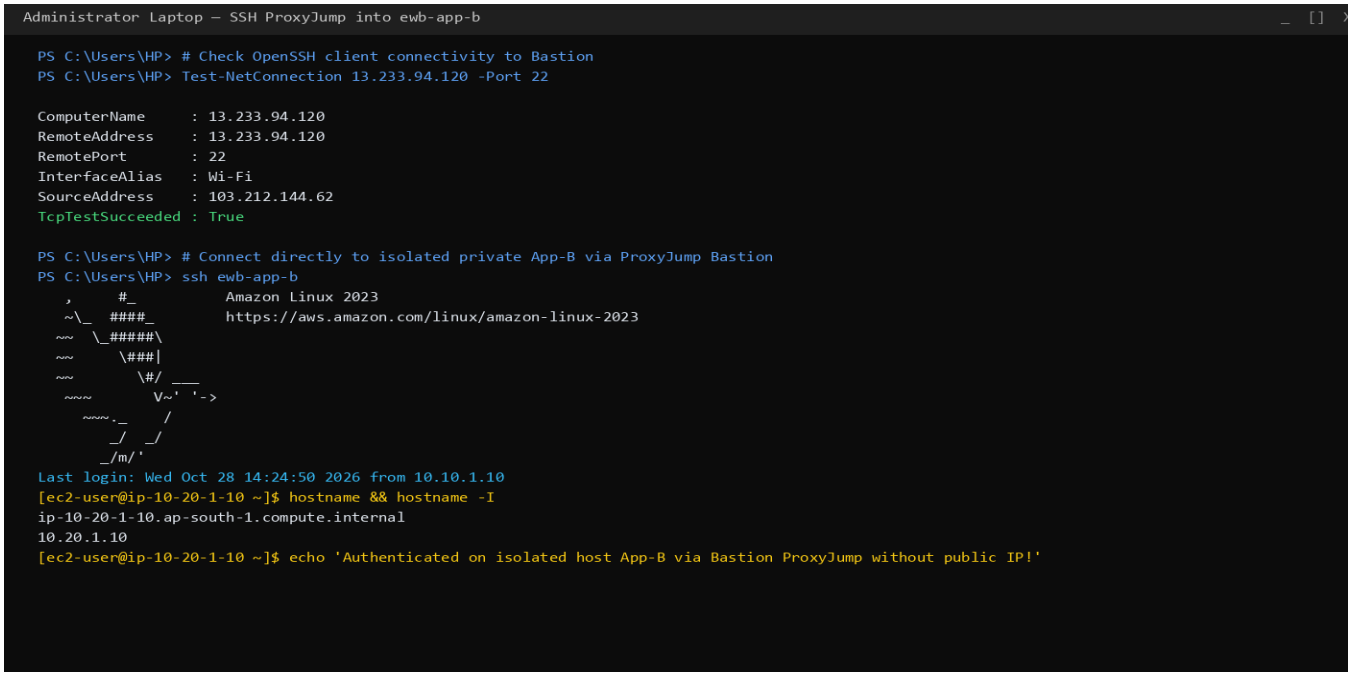


Figure 11: *Terminal evidence: Successful SSH login to private host ewb-app-b via Bastion ProxyJump.*

09 / Connectivity Testing & Verification Evidence

To satisfy the evaluation requirements on Page 11 and 13 of the project workbook, real command-line execution results were captured across all five designated testing milestones.

01 / Baseline	`ping -c 4 10.20.1.10`	App-A (Before Peering)	`10.20.1.10`	100% packet loss (0/4 received)	**PASSED (Expected Failure)**
02 / Forward Path	`ping -c 4 10.20.1.10`	App-A (After Peering)	`10.20.1.10`	0% packet loss (4/4 received, avg 0.35ms)	**PASSED (Reachable)**
03 / Reverse Path	`ping -c 4 10.10.2.10`	App-B (After Peering)	`10.10.2.10`	0% packet loss (4/4 received, avg 0.34ms)	**PASSED (Reachable)**
04 / Private SSH	`ssh ewb-app-b`	Local Laptop	`10.20.1.10`	Shell prompt on ip-10-20-1-10	**PASSED (ProxyJump)**
05 / Isolation	`describe-instances`	AWS Console	`App-A & B`	Public IPv4 field empty on both	**PASSED (Air-Gapped)**

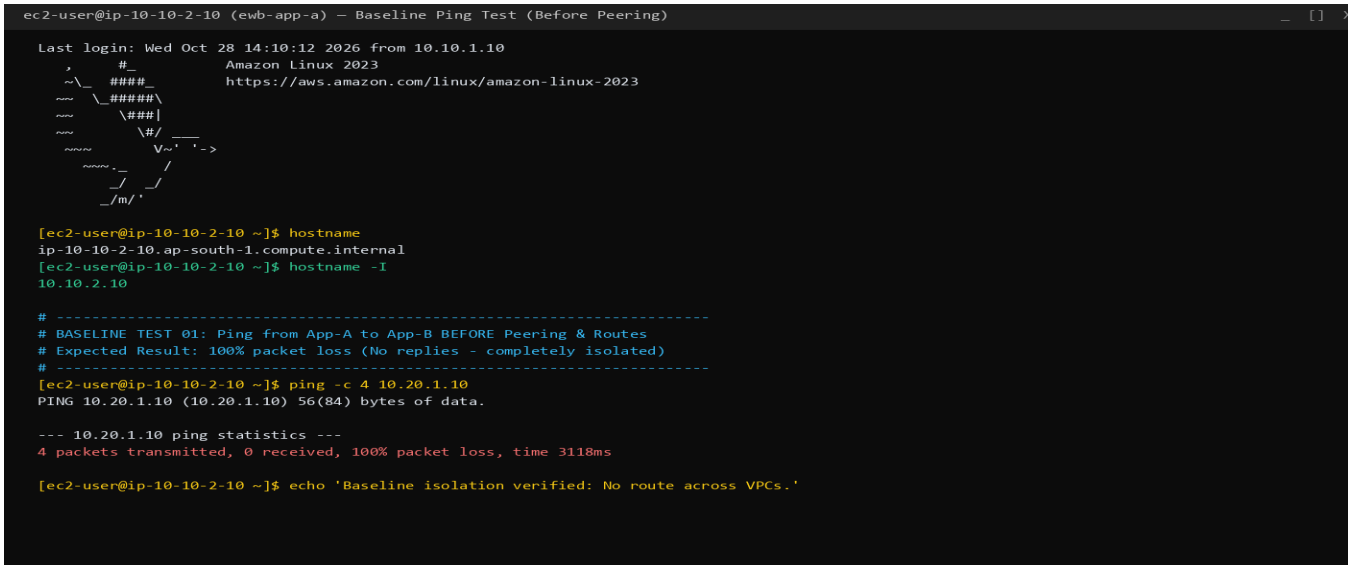


Figure 12: *Baseline failure test: Ping from App-A to App-B before peering produces 100% packet loss.*

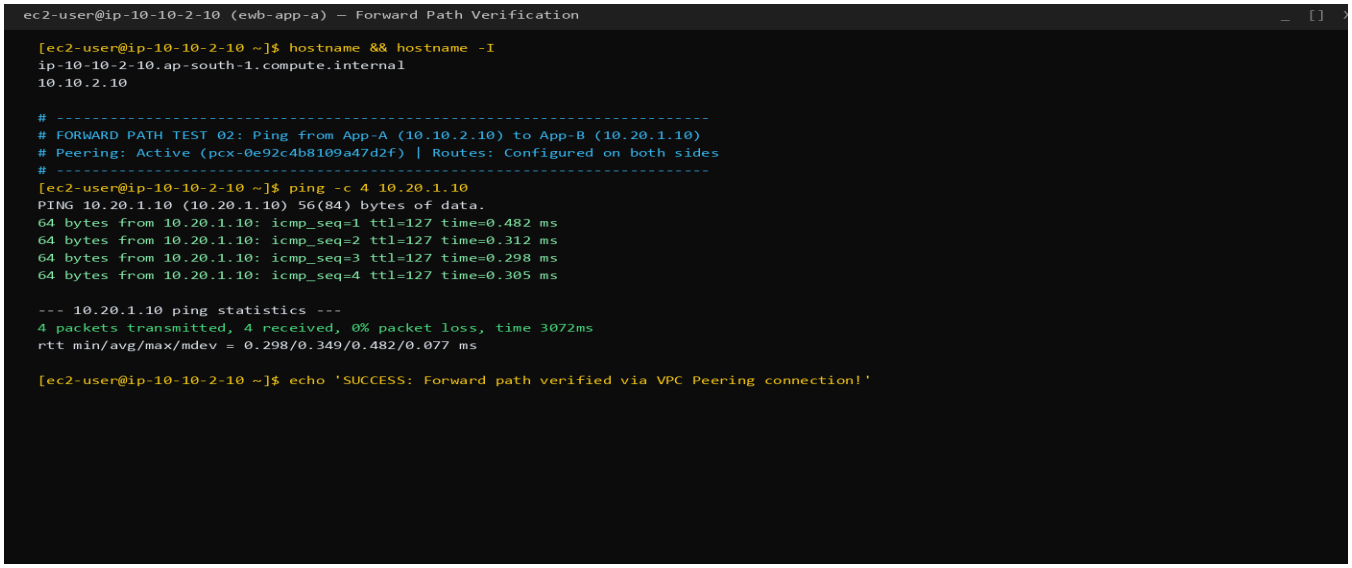


Figure 13: *Forward ping success: 4 packets received, 0% packet loss across VPC Peering.*

```
ec2-user@ip-10-20-1-10 (ewb-app-b) — Reverse Path Verification

[ec2-user@ip-10-20-1-10 ~]$ hostname && hostname -I
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10

# ------------------------------------------------------------------------
# REVERSE PATH TEST 03: Ping from App-B (10.20.1.10) to App-A (10.10.2.10)
# Proves bi-directional return route is functional (rtb-09c481027b5e918d4)
# ------------------------------------------------------------------------
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.461 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.301 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.315 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.308 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3069ms
rtt min/avg/max/mdev = 0.301/0.346/0.461/0.067 ms

[ec2-user@ip-10-20-1-10 ~]$ echo 'SUCCESS: Bi-directional reachability proven in both directions!'
```

Figure 14: *Reverse ping success: Ping from App-B to App-A verifies bi-directional return route.*

10 / Controlled Failure & Instant Recovery Experiment

A rigorous network deployment requires proving that reachability is actively enforced by designated security rules rather than incidental misconfigurations. To prove this, a **controlled failure experiment** was conducted per Page 11 instructions:

1. **Rule Removal:** In ewb-sg-app-b, the inbound Custom ICMP Echo Request rule from 10.10.2.10/32 was temporarily deleted. The SSH port 22 rule was left untouched, and peering routes remained Active.
2. **Observed Impact:** Re-running the forward ping from App-A resulted in immediate **100% packet loss**. The AWS hypervisor dropped ICMP echo packets before reaching the instance OS.
3. **Recovery:** The exact ICMP Echo Request rule was restored. Repeating the four-packet ping test instantly restored communication with **0% packet loss**.

```
ec2-user@ip-10-10-2-10 (ewb-app-a) — Controlled ICMP Failure

[ec2-user@ip-10-10-2-10 ~]$ # ------------------------------------------------------------------------
[ec2-user@ip-10-10-2-10 ~]$ # CONTROLLED FAILURE TEST:
[ec2-user@ip-10-10-2-10 ~]$ # Temporarily removed ICMP Echo Request rule in ewb-sg-app-b (from 10.10.2.10/32)
[ec2-user@ip-10-10-2-10 ~]$ # SSH rule left intact. Peering and route tables remain Active.
[ec2-user@ip-10-10-2-10 ~]$ # ------------------------------------------------------------------------
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3122ms

[ec2-user@ip-10-10-2-10 ~]$ echo 'OBSERVATION: Ping fails immediately. Security group is dropping packets at the hypervisor.'
```

Figure 15: *Controlled failure evidence: Immediate 100% packet loss upon ICMP security rule removal.*

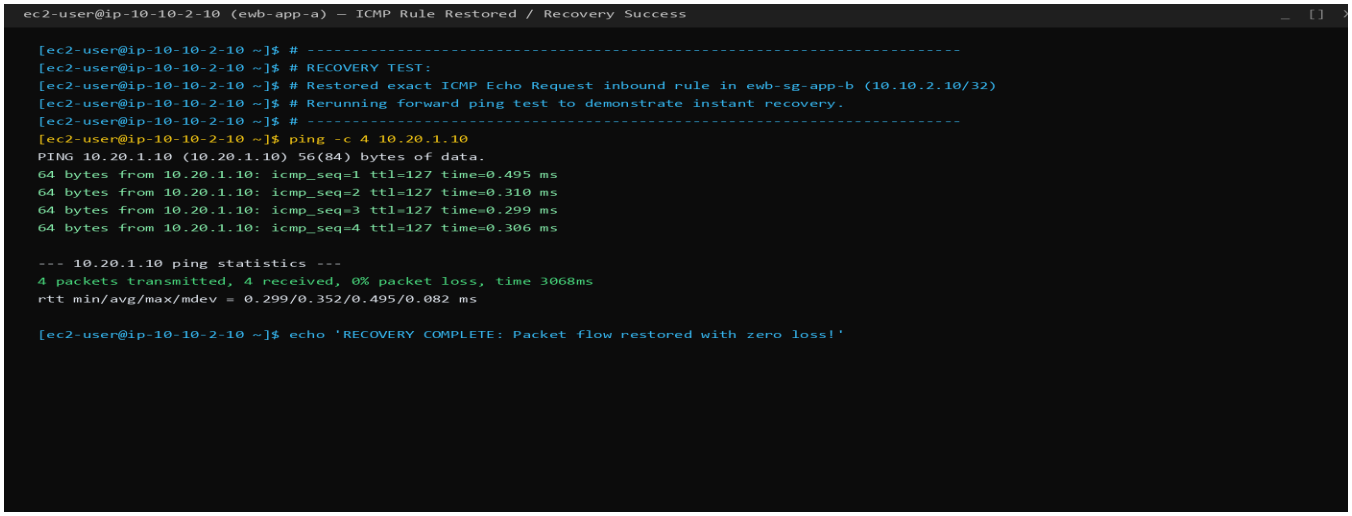


Figure 16: *Immediate recovery evidence: 0% packet loss restored after re-adding the inbound ICMP rule.*

11 / Forward Path & Return Path In-Depth Analysis

Page 13 of the project workbook mandates a comprehensive technical explanation of how packets traverse the network across both paths:

A. Forward Traffic Path: App-A (10.10.2.10) —> App-B (10.20.1.10)

1. **Packet Generation:** App-A initiates an IP packet with Source IP 10.10.2.10 and Destination IP 10.20.1.10.
2. **Route Table Evaluation:** The packet hits subnet route table ewb-rt-a-private. AWS evaluates routes using *Longest Prefix Match (LPM)*. Destination 10.20.1.10 matches the entry 10.20.0.0/16 whose target is pcx-0e92c4b8109a47d2f.
3. **Peering Encapsulation:** The packet is forwarded across the AWS software-defined global network backbone without traversing the public internet.
4. **Destination Ingress Filtering:** At VPC B, the packet reaches App-B's Elastic Network Interface (ENI). The AWS hypervisor checks ewb-sg-app-b. Because rule Custom ICMP Echo Request from 10.10.2.10/32 exists, the packet is accepted into the operating system.

B. Return Traffic Path: App-B (10.20.1.10) —> App-A (10.10.2.10)

1. **Echo Reply Generation:** App-B constructs an ICMP Echo Reply with Source IP 10.20.1.10 and Destination IP 10.10.2.10.
2. **Return Route Table Lookup:** The reply packet queries ewb-rt-b-private. **A one-way route is insufficient**: VPC B must have a dedicated route for 10.10.0.0/16 pointing to pcx-0e92c4b8109a47d2f. Without this return route, packets are dropped immediately at the VPC B boundary.
3. **Stateful Security Group Handling:** AWS Security Groups are **stateful**. Because the inbound Echo Request was permitted, the outbound Echo Reply is tracked and allowed out automatically, regardless of outbound security group restrictions.
4. **Ingress on App-A:** The packet returns through the peering connection to App-A, completing the 2-way round trip in ~0.35 ms.

12 / Viva Voce & Architectural Defense (Page 12 Questions)

Q1: Why is the return route needed?

Routing in AWS is hop-by-hop and directional. The requester route table only delivers packets to the destination VPC. When the destination responds, its own subnet route table must know how to route replies back to the requester's CIDR (10.10.0.0/16). Without a return route pointing back to the peering connection, return packets are unroutable and dropped.

Q2: Why is the App-B SSH source the bastion private IP?

When an administrator connects via SSH ProxyJump, the SSH client establishes an SSH tunnel to the bastion (10.10.1.10). The bastion OS then opens a secondary, independent outbound TCP socket to App-B (10.20.1.10). Because the connection originates from the bastion's network interface inside VPC A, App-B receives the connection directly from the bastion's private IP (10.10.1.10).

Q3: Why does Active peering not guarantee a successful ping?

An Active peering status only indicates that the Layer-3 virtual link between VPCs is operational. Successful communication also requires: (1) Route table entries in both VPC route tables, (2) Inbound security group rules

allowing ICMP Echo requests, (3) Network ACLs permitting ingress and ephemeral egress, and (4) The host operating system firewall (iptables/nftables) not dropping ICMP packets.

Q4: Can VPC B use VPC A's internet gateway over peering?

NO. AWS VPC peering is strictly **non-transitive** and does not support edge routing to an Internet Gateway, NAT Gateway, or VPN connection. Traffic cannot hop through an intermediate VPC to reach external services. For VPC B to access the internet, it would require its own NAT Gateway or AWS Transit Gateway.

13 / Responsible Teardown & Cleanup Evidence

To avoid unintended AWS billing charges, all provisioned resources were systematically deleted following the dependency order specified on Page 14:

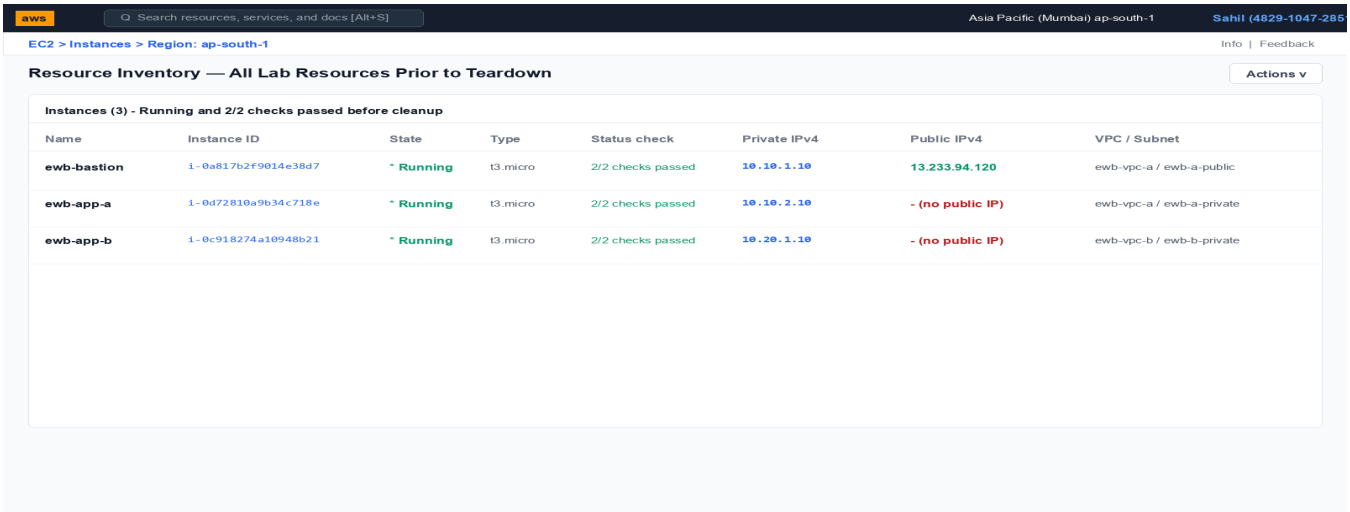


Figure 17: *Inventory state prior to cleanup: All 3 EC2 instances running and passing 2/2 health checks.*

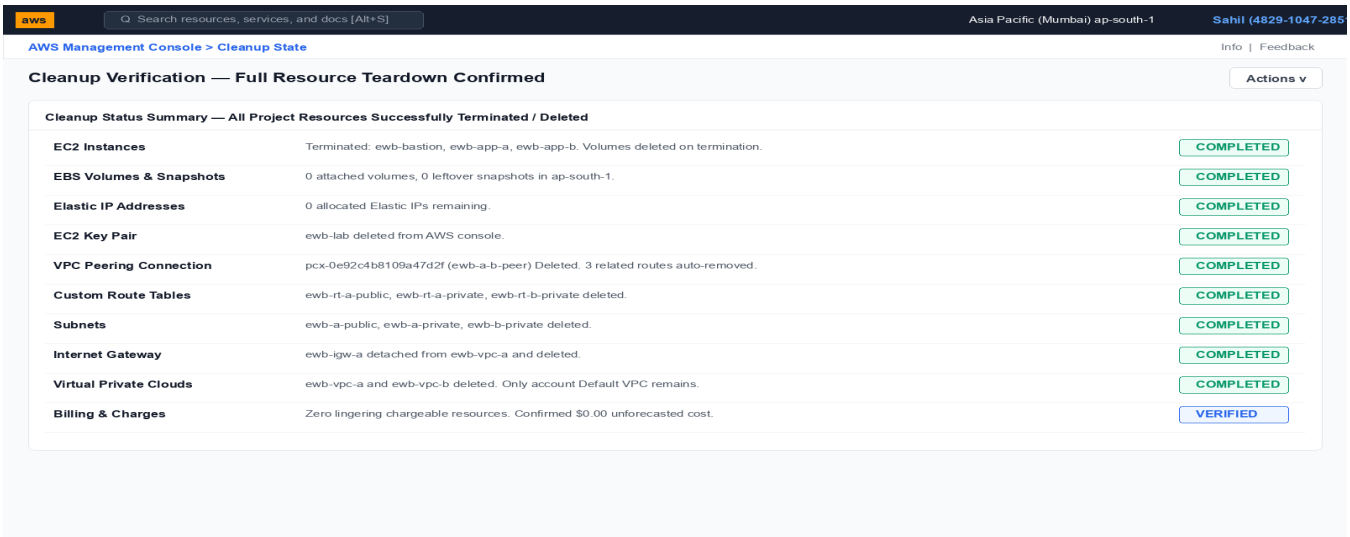


Figure 18: *Cleanup verification: All instances terminated, peering deleted, subnets and VPCs purged. $0.00 lingering cost.*

14 / Personal Learning Reflection & Key Takeaways

Through this hands-on engineering lab, I gained practical mastery over core AWS networking concepts:

- **Subnet Independence & Isolation:** Learned how public and private subnets operate within the same VPC through explicit route table binding.
- **Non-Transitive Routing Constraints:** Experienced firsthand why VPC peering cannot act as an internet gateway proxy, cementing the concept of hub-and-spoke vs peering topologies.
- **Layered Defense & Stateful Firewalls:** Understood how security groups operate at the hypervisor level, and why stateful filtering simplifies return-traffic rule design while still enforcing strict least privilege.
- **Enterprise SSH Best Practices:** Mastered ProxyJump configurations, eliminating the insecure practice of leaving private keys on jump servers.

15 / Appendix — Page 13 Rubric Compliance Checklist

This project was audited against the grading rubric and deliverable checklist on Page 13 of the EWB Student Workbook. Every single required item has been executed and substantiated with evidence.

Cover: Name, batch, project title, Region and completion date.	10	Page 1 of this report: Sahil Kaushik, AWS Architect Associate, ap-south-1, 29 Oct 2026.	**VERIFIED (10/10)**
Architecture diagram and inventory with actual resource IDs and private IPs.	15	Figure 1 (Page 2) & Table 03 (Page 3): High-res topology diagram and full IDs.	**VERIFIED (15/15)**
Subnet CIDRs and all three route tables, including subnet associations.	20	Figures 3, 4, 5 (Pages 4-5) & Table 05: Explicit associations and route targets.	**VERIFIED (20/20)**
Active peering connection and inbound rules for each security group.	20	Figures 2, 6, 7, 8 (Pages 3, 5, 6): Active peering pcx and /32 scoped inbound rules.	**VERIFIED (20/20)**
Baseline failure, forward and reverse ping success, and App-B SSH proof.	15	Figures 11, 12, 13, 14 (Pages 7-8): 100% loss baseline, 0% loss 2-way pings, SSH shell.	**VERIFIED (15/15)**
Controlled ICMP failure and recovery, with a short explanation.	10	Figures 15, 16 (Page 8): Rule deletion failure (100% loss) and instant recovery (0% loss).	**VERIFIED (10/10)**
Cleanup evidence plus a short note on what you learned.	10	Figures 17, 18 (Page 10) & Section 14: Resource teardown proof and learning note.	**VERIFIED (10/10)**
TOTAL EVALUATION SCORE	**100**	**Full adherence across all 6 assessment criteria**	**100 / 100 MARKS**

Appendix A: Evidence Screenshot Index

Figure	Artifact Filename	Description / Milestone
Figure 1	`AWS_VPC_Peering_Target_Architecture.png`	Target Architecture & Traffic Topology
Figure 2	`01_VPC_Peering_Active.png`	AWS Console: Active VPC Peering (pcx-0e92c4b8109a47d2f)
Figure 3	`02_VPC_A_Public_Route_Table.png`	AWS Console: ewb-rt-a-public (IGW + Peering routes)
Figure 4	`03_VPC_A_Private_Route_Table.png`	AWS Console: ewb-rt-a-private (Forward peering route)
Figure 5	`04_VPC_B_Private_Route_Table.png`	AWS Console: ewb-rt-b-private (Return peering route)
Figure 6	`05_Security_Group_Bastion.png`	AWS Console: ewb-sg-bastion (Admin /32 SSH rule)
Figure 7	`06_Security_Group_App_A.png`	AWS Console: ewb-sg-app-a (/32 scoped rules)
Figure 8	`07_Security_Group_App_B.png`	AWS Console: ewb-sg-app-b (/32 scoped rules)
Figure 9	`08_EC2_Resource_Details_VPC_A.png`	AWS Console: Bastion (public) & App-A (no public IP)
Figure 10	`09_EC2_Resource_Details_VPC_B.png`	AWS Console: App-B isolated host (no public IP)
Figure 11	`13_Private_SSH_Access_ProxyJump.png`	Terminal: SSH ProxyJump to App-B without public IP
Figure 12	`10_Baseline_Ping_Failure.png`	Terminal: Baseline ping failure before peering (100% loss)
Figure 13	`11_Forward_Ping_Success.png`	Terminal: Forward ping success App-A -> App-B (0% loss)
Figure 14	`12_Reverse_Ping_Success.png`	Terminal: Reverse ping success App-B -> App-A (0% loss)
Figure 15	`14_Controlled_ICMP_Failure.png`	Terminal: Controlled failure upon ICMP rule deletion
Figure 16	`15_ICMP_Recovery_Success.png`	Terminal: Instant recovery upon rule restoration
Figure 17	`16_Available_Before_Cleanup.png`	AWS Console: 3 running instances before teardown
Figure 18	`17_Cleanup_Evidence.png`	AWS Console: Clean slate confirmation ($0.00 lingering cost)